# CodeLens AI: The Complete Engineering Report

## A Comprehensive Guide to Architecture, Technologies, and Deployment

---

This document serves as the complete technical book for CodeLens AI. It is designed to take a reader from knowing absolutely nothing about the project to fully understanding its architecture, how the code works, and how to deploy it to production.

---

# Chapter 1: Introduction to CodeLens AI

## 1.1 What is CodeLens AI?

CodeLens AI is a modern Software-as-a-Service (SaaS) application designed to automate the process of code reviews. Instead of waiting hours or days for a human senior engineer to review a Pull Request, developers can paste their code or link a GitHub repository to CodeLens AI and receive instant, actionable feedback.

## 1.2 The Problem It Solves

In modern software development, code reviews are a massive bottleneck. Senior engineers spend up to 20% of their time reviewing the code of junior colleagues. Furthermore, human reviewers often miss subtle security vulnerabilities, such as an obscure SQL injection vector or a Cross-Site Scripting (XSS) vulnerability.

CodeLens AI acts as a tireless, ultra-fast Senior Staff Engineer. It never sleeps, and it checks for best practices, performance bottlenecks, and security flaws in seconds.

## 1.3 Key Value Propositions

1. **Speed**: Reviews take 3-5 seconds, compared to hours for a human.
2. **Security**: Integrated OWASP Top 10 vulnerability scanning.
3. **Education**: It doesn't just tell the developer the code is wrong; it provides a detailed explanation of *why* it is wrong and offers a refactored solution.
4. **Seamless Integration**: Directly connects to GitHub to scan Pull Requests automatically.

---

# Chapter 2: The Technology Stack

To build a modern, scalable, and secure web application, we carefully selected a specific set of technologies. We refer to this as the "Tech Stack."

## 2.1 The Frontend Stack

The frontend is the visual part of the website that users interact with.

- **Framework - Next.js 14:** We chose Next.js (specifically the App Router) because it is the industry standard for building React applications. It provides server-side rendering (SSR), which makes the website incredibly fast and great for Search Engine Optimization (SEO).

- **Core Library - React:** React allows us to build the UI using reusable "components" (like buttons, sidebars, and forms).
- **Language - TypeScript:** TypeScript adds strict type definitions to JavaScript. This prevents thousands of potential bugs before the code is even run.
- **Styling - Custom CSS Variables:** Instead of a heavy UI library, we built a bespoke "Glassmorphism" design system using pure CSS.
- **Authentication - NextAuth.js:** A complete open-source authentication solution for Next.js applications that securely handles login sessions.
- **Code Editor - Monaco Editor:** This is the exact same underlying technology that powers Visual Studio Code, giving our users a familiar, powerful text area to paste their code.

## 2.2 The Backend Stack

The backend is the invisible engine that processes data, talks to the database, and handles the AI logic.

- **Runtime - Node.js:** Node.js allows us to run JavaScript outside the browser, directly on the server.
- **Framework - Express.js:** Express is a fast, unopinionated web framework for Node.js. It handles incoming HTTP requests (like "Create a User" or "Analyze Code") and sends back JSON responses.
- **Language - JavaScript:** The backend is written in modern ES6+ JavaScript.
- **Authentication - JSON Web Tokens (JWT) & bcrypt:** We use bcrypt to irreversibly hash user passwords, and JWTs to securely prove a user's identity on every API request.
- **CORS Management:** Cross-Origin Resource Sharing (CORS) is configured to ensure only our specific frontend (Vercel) can talk to our backend API.

## 2.3 The Database & Data Layer

- **Relational Database - PostgreSQL:** Postgres is the world's most advanced open-source relational database. It stores our users, their reviews, and connected repositories in strict tables.
- **Object Relational Mapper (ORM) - Prisma:** Writing raw SQL queries is error-prone. Prisma acts as a bridge; we write clean JavaScript code, and Prisma translates it into highly optimized SQL commands.

## 2.4 The AI Engine

- **Provider - Google Gemini:** We use Google's Gemini Flash AI model. We chose Gemini Flash because it offers the perfect balance of blazing speed and deep analytical reasoning required for code reviews.

---

# Chapter 3: High-Level System Architecture

How does everything talk to each other? CodeLens follows a strictly decoupled Client-Server architecture.

## 3.1 The Client (Frontend)

The frontend, hosted on Vercel, is purely a presentation layer. It never directly accesses the database. Instead, when a user clicks a button (e.g., "Login"), the frontend formats a piece of data (a JSON object) and sends it over the internet via an HTTP request.

## 3.2 The Server (Backend API)

Hosted on Render, the backend waits for these HTTP requests.

1. **Routing:** Express looks at the URL (e.g., `/api/auth/login`) and routes it to the correct controller function.
2. **Middleware:** Before the controller runs, middleware steps in. For example, the `authMiddleware` checks if the user provided a valid JWT. If not, it rejects the request immediately.
3. **Controller:** The controller executes the business logic (e.g., verifying a password).
4. **Data Access:** If data is needed, the controller asks Prisma to fetch it from the PostgreSQL database.
5. **Response:** Finally, the controller packages the data back into a JSON response and sends it to the frontend.

## 3.3 The Authentication Bridge (The Hardest Part Explained)

One of the most complex architectural challenges was combining NextAuth (a frontend-heavy auth system) with a custom Express backend.

Here is exactly how it works:

1. User clicks "Login with GitHub" on the Next.js frontend.
2. NextAuth redirects them to GitHub. The user approves the login.
3. GitHub sends NextAuth a basic profile (email, name, picture).
4. **The Bridge:** Inside the NextAuth configuration, we wrote a special `jwt` callback interceptor. Before NextAuth finishes logging the user in, it takes that GitHub profile and secretly sends it to our Express backend ( `POST /api/auth/github` ).
5. The Express backend looks in the database: "Do I know this GitHub user?" If no, it creates a new row in the PostgreSQL database. If yes, it retrieves their data.
6. Express then creates a secure JWT token and sends it back to NextAuth.
7. NextAuth takes that backend token and bakes it into the browser's secure HttpOnly session cookie.
8. Now, whenever the frontend needs to fetch private data, it retrieves that token from the session and adds it to the `Authorization` header.

---

# Chapter 4: Deep Dive into Code Reviews (The AI Logic)

How does the website actually review code? This is where the magic happens.

## 4.1 The Prompt Engineering

When a user pastes code and clicks "Review", the frontend sends the code string to the backend. The backend doesn't just forward it to Google Gemini; it surrounds it with a meticulously crafted "System Prompt".

We instruct Gemini to adopt a specific persona: `"You are a Senior Staff Software Engineer and Security Expert. You are conducting a rigorous code review..."`

We also heavily constrain the output. We tell Gemini it *must* respond in perfectly valid JSON, matching a strict schema. This is crucial because our frontend needs structured data to draw the charts and lists; it cannot parse a standard conversational chat.

## 4.2 The Expected JSON Schema

We force the AI to return data in this exact shape:

```json
{
  "score": 85,
  "issues": [
    {
      "type": "security",
      "severity": "critical",
      "line": 42,
      "description": "SQL Injection vulnerability found in user input handling",
      "suggestion": "Use parameterized queries or an ORM like Prisma"
    }
  ],
  "metrics": {
    "cyclomaticComplexity": 14,
    "linesOfCode": 120
  }
}
```

### 4.3 Parsing and Fallbacks

AI models can sometimes hallucinate or return invalid JSON. To protect the application from crashing, the backend wraps the AI call in a `try/catch` block. It attempts to strip out markdown formatting (like `json ...` ) and `JSON.parse` the result. If the AI fails to generate metrics, the backend has JavaScript fallback functions that manually count the lines of code to ensure the dashboard always has data to display.

---

## Chapter 5: Designing a Premium SaaS Experience (Frontend UI)

A modern AI product must look exceptionally premium to instill trust. CodeLens AI achieves this without using bulky UI libraries like Bootstrap or Material UI.

### 5.1 The Glassmorphism Aesthetic

The entire UI is built on a concept called "Glassmorphism." This involves mimicking the look of frosted glass. We achieve this using specific CSS properties:

- `background: rgba(255, 255, 255, 0.03)` (A very sheer, transparent white)
- `backdrop-filter: blur(12px)` (This blurs whatever is behind the element, creating the frosted glass effect)
- `border: 1px solid rgba(255, 255, 255, 0.08)` (A crisp, subtle edge)

### 5.2 Animations and Perceived Performance

If an AI review takes 4 seconds, a static loading spinner feels broken. Instead, we use psychological design tricks:

1. **Shimmer Effects:** "Skeleton" shapes are displayed where data will eventually load. A CSS gradient sweeps across these skeletons, making the app feel alive and "thinking."

2. **Staggered Entrances:** When a page loads, elements don't appear all at once. Using CSS keyframes and `animation-delay`, the title fades in, then the charts slide up 0.1 seconds later, then the lists appear. This creates a deeply satisfying, fluid user experience.

### 5.3 Route Protection (Guarding the Gates)

If an unauthenticated user tries to type `codelens.com/dashboard` into their URL bar, they should not see the dashboard. In Next.js, we enforce this using the `useSession()` hook. Every protected page has a layout check:

```
const { status } = useSession();
const router = useRouter();

useEffect(() => {
    if (status === "unauthenticated") {
        router.push("/login"); // Kick them out instantly
    }
}, [status])
```

---

## Chapter 6: The Database Schema (PostgreSQL)

Our PostgreSQL database, managed entirely by Prisma, has four core models (tables).

### 6.1 The User Model

Stores the identity of the person logging in.

- `id` : Unique identifier (UUID).
- `email` : Required for login and notifications.
- `password` : Hashed string (nullable for OAuth users).
- `githubId` : Tracks connected social accounts.
- `plan` : Enum (FREE, PRO, ENTERPRISE) for monetization.

### 6.2 The Review Model

Stores the history of every code analysis performed.

- `id` : Unique identifier.
- `userId` : A Foreign Key linking the review back to the User who created it. This forms a One-to-Many relationship (One User has Many Reviews).
- `code` : The raw string of code uploaded.
- `score` : The AI-generated quality score (0-100).
- `analysis` : JSON column storing the structured AI feedback (issues, suggestions).

### 6.3 Repositories & Teams

Future-proofing models that allow users to link entire GitHub repositories for automated Pull Request scanning, and create Teams for collaborative enterprise environments.

---

# Chapter 7: The Journey of a Request

Let's trace the exact path of data when a user clicks "Review Code".

1. **User Action:** The user pastes Python code into the Monaco Editor and clicks submit.
2. **Frontend Interceptor:** Axios intercepts the request, grabs the underlying `session.accessToken`, and attaches it.
3. **The Network:** The JSON payload travels from the Vercel edge network to the Render backend server.
4. **Backend Middleware:** Express receives it. `authMiddleware` decodes the token, confirming User ID #452 is making the request.
5. **Controller Routing:** The router passes the request to the `createReview` controller.
6. **AI Service:** The controller calls `ai.service.js`, which packages the Python string into the Gemini Prompt.
7. **External API:** Our backend talks to Google's servers over HTTPS.
8. **Processing:** Gemini analyzes the Python code, generating a structured JSON critique.
9. **Database Write:** Express takes Google's JSON, and asks Prisma to `prisma.review.create()`, saving it permanently to Postgres.
10. **The Return:** Express sends an HTTP 200 OK response back to Vercel with the newly saved review.
11. **Frontend Update:** React state updates, the loading skeleton vanishes, and the beautiful glassmorphism chart animates up to show a score of "85/100".

---

# Chapter 8: The Comprehensive Deployment Guide

Building an app on a local laptop is only 50% of the job. Putting it on the global internet with domains and SSL certificates is the other half. Here is exactly how CodeLens is deployed.

## 8.1 Database Hosting

We cannot run a database on our laptop for production. We use a cloud provider like **Neon** or **Aiven**.

1. Create a free PostgreSQL cluster.
2. The provider gives us a "Connection String" (e.g., `postgresql://user:pass@ep-blue-sky.us-east-1.aws.neon.tech/main`).
3. This string allows our backend to talk to the database securely over the internet.

## 8.2 Backend Deployment (Render)

Render is a Platform-as-a-Service that automatically builds and runs Node.js servers.

1. We connect Render to our GitHub repository.
2. We tell Render the build command: `npm install && npx prisma generate` (this sets up dependencies and builds the Prisma client).
3. We tell Render the start command: `node src/server.js`.
4. **Critical Step:** We must inject "Environment Variables" into the Render dashboard. We input the `DATABASE_URL`, a secret `JWT_SECRET`, the `GEMINI_API_KEY`, and crucially, the `FRONTEND_URL`.
5. When code is pushed to the `main` branch, Render detects it, pulls it, and restarts the server silently.

## 8.3 Frontend Deployment (Vercel)

Vercel are the creators of Next.js, making them the absolute best platform for hosting our frontend.

1. Connect Vercel to the GitHub repository.
2. Vercel automatically detects it's a Next.js app and configures the builds.
3. **Critical Step:** Environment Variables. The frontend needs to know where the backend lives. Instead of hardcoding `localhost:5000` into the JavaScript, we define `NEXT_PUBLIC_API_URL`. In production, we set this to `https://code-reviwer.onrender.com/api`.
4. We also set `NEXTAUTH_URL` to our Vercel domain, and inject our `GOOGLE_ID` and `GITHUB_ID` secrets so social logins work.
5. Vercel compiles the React code into optimized, minified static files and deploys them to hundreds of edge servers worldwide via a CDN (Content Delivery Network).

## 8.4 The CORS Security Model

One major hurdle in deployment is CORS (Cross-Origin Resource Sharing). Browsers have a strict security rule: if a user is visiting `frontend.com`, that website's JavaScript is not allowed to quietly make requests to `backend.com` unless `backend.com` explicitly grants permission.

In our backend `server.js`, we use the `cors` package:

```
app.use(cors({
  origin: process.env.FRONTEND_URL, // e.g., https://code-reviwer-three.vercel.app
  credentials: true
}));
```

This tells the browser, "Yes, I know this frontend, they are allowed to talk to me."

---

# Chapter 9: Future Roadmap and Scaling

CodeLens AI is currently a robust baseline architecture. Here is how it can evolve into an enterprise product.

## 9.1 Scaling the AI

Currently, the user's browser waits while Gemini processes the code. If the code is massive (50,000 lines), the HTTP request will time out (take longer than 30 seconds). **Solution:** A Pub/Sub Message Queue (like Redis or RabbitMQ). When a user submits large code, the backend instantly returns "Review Pending". A separate background worker picks up the job, talks to Gemini, and updates the database. The frontend uses WebSockets or Long Polling to detect when the review is finished and alerts the user.

## 9.2 GitHub App Integration

Instead of users pasting code, CodeLens will become a registered "GitHub App". When a developer opens a Pull Request on their company GitHub, GitHub will send a "Webhook" (an automated HTTP POST request) to our Express Backend. Our backend will fetch the changed files, run them through Gemini, and use the GitHub API to leave actual line-by-line comments directly on the GitHub UI, just like a human engineer would.

### 9.3 Monetization Engine

Adding Stripe integration for the billing page. The backend auth middleware will be updated to check `user.plan`. Free users can run 5 reviews a month. Pro users ($15/mo) get unlimited reviews and earlier access to new AI models.

---

## Summary and Conclusion

CodeLens AI represents the bleeding edge of modern web development. It perfectly synchronizes a blistering fast React frontend (Next.js) with a secure, highly-customized Node.js API (Express), backed by rigid relational data structures (PostgreSQL/Prisma), all while orchestrating complex interactions with the world's most advanced Artificial Intelligence models (Google Gemini).

The decoupled architecture specifically protects the system from scaling bottlenecks, allowing the frontend to be distributed via CDN globally, while the heavy-lifting AI backend can scale its compute resources independently. CodeLens is not just an application; it is a blueprint for the future of AI-augmented Developer Operations (DevOps).